

Part Two: Exercises

Computing without a Computer

MAP PIECE

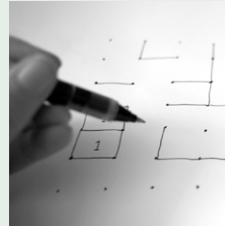
Draw a map to get lost.

1964 spring

Ono's Grapefruit

Browse the prompts in Yoko Ono's *Grapefruit* series, and, if possible, execute one.¹

Contemplate giving instructions as a mode of creative practice. Devise an instructional painting in Ono's style.

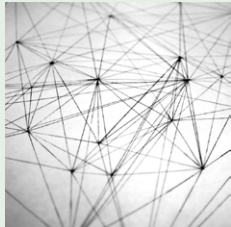


Drawing Games

In pairs, play a game of *Dots and Boxes* (by Édouard Lucas) and *Sprouts* (by John H. Conway and Michael S. Paterson) to deepen your understanding of rule-based drawing games.⁴

Wall Drawing #118

Execute Sol LeWitt's *Wall Drawing #118* (1971): "On a wall surface, any continuous stretch of wall, using a hard pencil, place fifty points at random. The points should be evenly distributed over the area of the wall. All of the points should be connected by straight lines."²

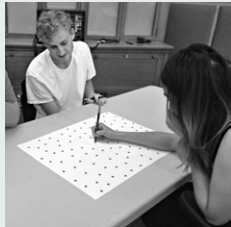


Zoom Schwartz Profigliano

In groups of five, play the rule-based conversation game "Zoom Schwartz Profigliano," in which an ever-expanding vocabulary of whimsical nonsense words establishes precise rules for what can be spoken, to whom, when, and how.⁵ (Photo: College of DuPage.)

Conditional Design: The Beach

Organize into groups of four, and give each person their own color marker. Execute "The Beach" from the "Conditional Design Manifesto" by Luna Maurer et al.: "Each turn, find the most empty space on the paper and place a dot in the middle of it."³

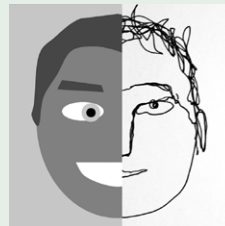
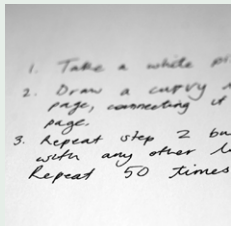


Be the Computer I

Arrange the class in a grid configuration and give each person a sheet of paper so that each student is responsible for a pixel. One person takes charge and programs the pixels by showing the group a script or by giving them direct commands.⁶

Procedural Drawing

Develop your own procedural drawing rule set in the spirit of Sol LeWitt or the prompts from conditionaldesign.org. Have one or more of your peers produce a drawing with your system. Consider devising systems using other materials such as tape or string.



Be the Computer II

Write a simple program to create a static drawing. Give your code to a peer, and (without showing them your screen) ask them to predict and hand-draw the result. When they are finished, compare the computer's drawing with their hand-drawn work.



Human Fax Machine

In groups of two or four, devise a sound language for describing how marks are made. You may use a sound-making device (two spoons, a set of keys) or your own mouth noises, so long as you use no spoken words. Write down the code, then split your group into “transmitters” and “receivers,” with a visual barrier in between. Test the system. A “transmitter” should take a simple hand-drawn image and transmit it across the barrier to the “receiver.” When finished, compare the original with the transmitted image and fix any problems in your system. Do this with several images, and discuss.⁷

Additional References

Casey Reas, *[Software] Structures*, 2004, <http://artport.whitney.org/commissions/softwarestructures/text.html>.

Basil Safwat, *Processing.A4*, 2013, <http://www.basilsafwat.com/projects/processing.a4/>.

FoAM, notes for Mathematick Arts workshop, 2011, https://libarynth.org/mathematick_arts_2011.

J. Meejin Yoon, “Serial Notations / Drift Drawings,” 2003, <https://ocw.mit.edu/courses/architecture/4-123-architectural-design-level-i-perceptions-and-processes-fall-2003/assignments/problem1.pdf>.

Notes

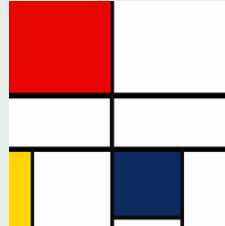
1. Yoko Ono, *Grapefruit* (London: Simon & Schuster, 2000).
2. Andrew Russeth, “Here Are the Instructions for Sol LeWitt’s 1971 Wall Drawing for the School of the MFA Boston,” *Observer*, October 1, 2012, <https://observer.com/2012/10/here-are-the-instructions-for-sol-lewitts-1971-wall-drawing-for-the-school-of-the-mfa-boston/>.
3. Luna Maurer, Edo Paulus, Jonathan Puckey, and Roel Wouters, “Conditional Design: A Manifesto for Artists and Designers,” accessed April 14, 2020, <https://conditionaldesign.org/workshops/the-beach/>.
4. Wikipedia, “Dots-and-Boxes,” http://en.wikipedia.org/wiki/Dots_and_Boxes; Wikipedia, “Sprouts,” [http://en.wikipedia.org/wiki/Sprouts_\(game\)](http://en.wikipedia.org/wiki/Sprouts_(game)).
5. David King, “Zoom Schwartz Profigliano,” 1998, <https://www.scottpages.net/ZSP-Rules-2012.pdf>.
6. John Maeda, *Creative Code* (New York: Thames and Hudson, 2004), 216.
7. Brogan Bunt and Lucas Ihlein, “The Human Fax Machine Experiment,” *Scan (Sydney): Journal of Media Arts Culture* 10, no. 2 (2013): 1–26.

Graphic Elements



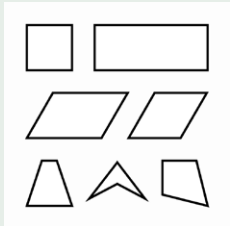
One with Everything

Explore your graphics toolset by drawing one of each type of primitive it provides. For example, you might draw a rectangle, ellipse, arc, line segment, Bézier curve, polyline, and polygon. Experiment with their options and parameters, such as fill color, stroke weight, etc.



Coding Mondrian

Using code, reproduce a Mondrian painting, such as *Composition No. III, with Red, Blue, Yellow and Black* (1929). Pay attention to detail.



Quadrilateral Zoo

Write commands to plot the vertices of a family of quadrilaterals: square, rectangle, parallelogram, rhombus, trapezoid, dart, and kite.



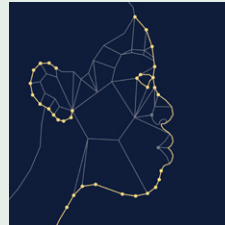
Coding Stadia II

Select and crop a small rectangular region from Julie Mehretu's painting *Stadia II*. Using a program such as Photoshop, read out the colors and coordinate data from this fragment. Use this data to faithfully recreate the fragment with shapes, lines, curves, and custom shape functions.



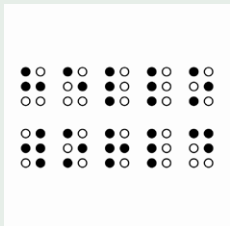
Draw Your Initials

Draw your initials with primitive shapes and lines.



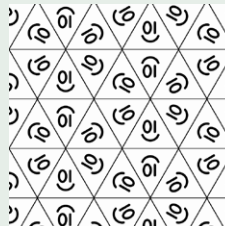
Draw, Then Code

Spend 20 minutes drawing on paper: a self-portrait, landscape, still life, or geometric design. Create your drawing with adequate care and detail. Now recreate your drawing using code. (Image: *p5.js Self-Portrait*, a student project by Zainab Aliyu, 2015.)



Braille Tool

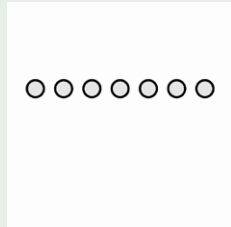
Reproduce the Braille alphabet using filled and unfilled circles. If you can, store a representation of these patterns in an array, and create a tool that allows a user to compose, print, and (with a stylus) emboss Braille messages.



Kaleidoscope

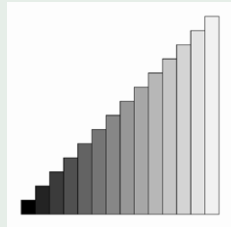
Devise a small graphical motif. Write a program that uses your toolset's graphics transforms to translate, reflect, and rotate copies of this motif, in the manner of a kaleidoscope.

Iteration



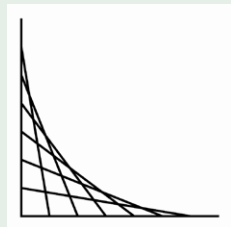
Simple Iteration: Seven Circles

Use iteration to copy the figure on the left, in which seven circles are positioned across the canvas. The position of each circle should be computed using your loop's counting variable. Make sure the first circle is inset by a margin; it should not lie on the edge of the canvas.



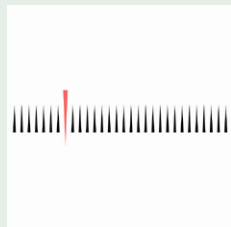
Transitioning Rectangles

Recall that *any* visual property can be linked to a loop variable, not just position. Use iteration to generate a series of rectangles. Your code should simultaneously control several of the rectangles' visual properties, including their position, height, and fill color.



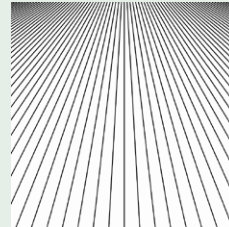
String Art Challenge

Use iteration to recreate the figure on the left. Your code should draw exactly eight lines.



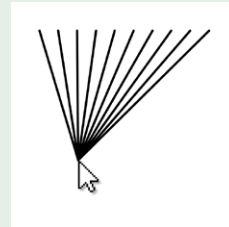
Mini-Calendar

Use iteration to render a row of visual elements: one element for each of the days of the current month. All of the elements should be drawn identically, *except* for the one whose index corresponds to the current day. Differentiate this element in some way.



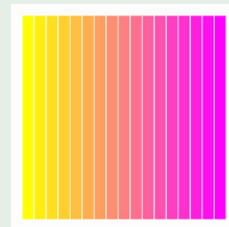
Receding Landscape

Use iteration to create a series of vertical lines across the screen. Create the illusion of a receding landscape by placing the lines' endpoints more closely together at the top of the canvas.



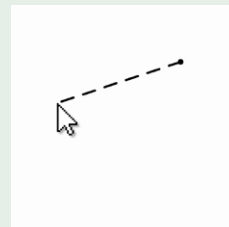
Lines to the Cursor

Use iteration to create an interactive display featuring a series of ten lines. Each line should connect the cursor to one of a series of points distributed evenly across the canvas.



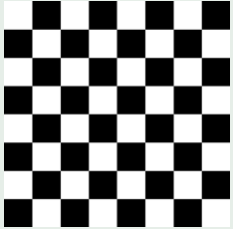
Color-Bar Gradient

Using iteration, generate a gradient that "lerps" (linearly interpolates) from one fill color to another across exactly 17 rectangles. Implement some code that randomizes the two terminal colors whenever the user clicks a button.



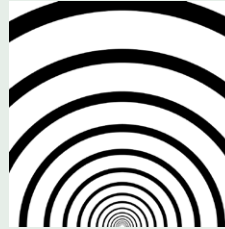
Dashed Line

Write a program that generates a dashed line between two points. (You may not use a "ready-made" dashed line.) Your dashes should always have a fixed length, such as 10 pixels, so that longer lines require more dashes. Connect one of your line's endpoints to the cursor.



Nested Iteration: Checkers

Create a checkerboard using a nested loop. Recall that a checkerboard is an 8x8 grid of alternating black and white squares, starting with white in the top left corner.



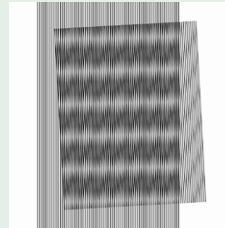
Geometric Progression

Generate a series of visual elements whose dimensions exhibit a geometric progression. The size of each element should be computed by multiplying the size of the one before it by a constant ratio. In the design at left, each circle is 1.3 times larger than the previous one inside it.



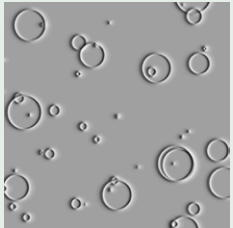
Iteration with Functions

Write a function that encapsulates the code to render a simple visual element (a leaf, face, etc.). Give your function arguments that determine where the element will be positioned. Using iteration, call your function to display a grid of these elements.



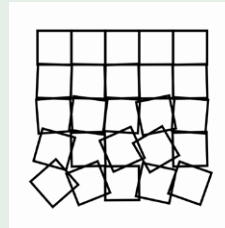
Moiré Patterns

Generate a set of parallel lines or curves, spaced at narrow intervals. Overlap two copies of your line set, differing by a small rotation, to create a moiré pattern. Place some dimension of variability, such as the line separation or rotation angle, under interactive control.



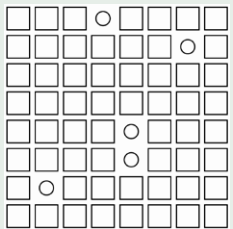
Stochastic Elements

Create an evocative composition in which an iterative loop deposits small elements in random locations around the canvas. These elements might look like craters, potholes, pimples, ants, chocolate chips, holes in Swiss cheese...



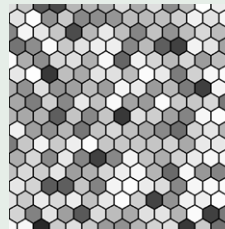
Recoding *Schotter*

Schotter by Georg Nees (1968) is a classic algorithmic artwork that depicts a gradient from order to chaos across a 12x22 grid of squares. In it, the squares' orientations become increasingly randomized towards the bottom of the page. Re-code this work, paying attention to detail.



Interrupted Grid

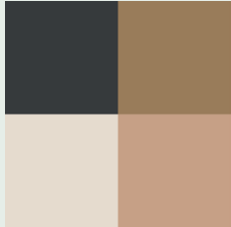
Use nested iteration to generate a grid of visual elements. Write code such that on each iteration, with a small random probability, an alternative element is occasionally drawn instead.



Hexagonal Grid

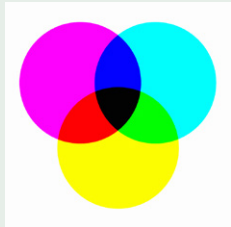
Write a custom function that draws a hexagon. Using iteration, call this function to fill your sketch with a grid of hexagons. For regular hexagons, you may need to consult some trigonometry.

Color



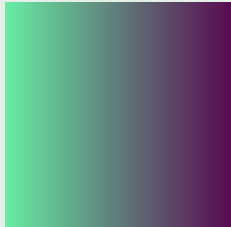
Color Observation

Carefully observe the color of your shirt, your table, the wall of your room, and the palm of your hand. Proceeding exclusively by modifying numbers in code, and without using a camera or scanning device, reproduce these colors to the best of your ability.



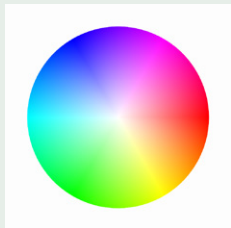
Overlapping Color

Overlap three semi-transparent circles, each with a different color, to create regions of overlapping colors. Explore the effects of different colors and transparency values. Experiment with different pixel transfer modes (also called “blend modes”). Draw the circles without outlines.



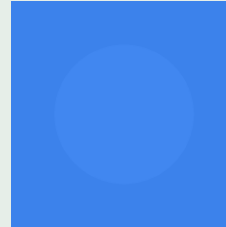
Constructing a Gradient

Create a smooth gradient between two colors. *Hint:* use iteration to render many thin, adjacent parallel lines, each with subtly different colors. How does your choice of color model (RGB or HSB) affect the result?



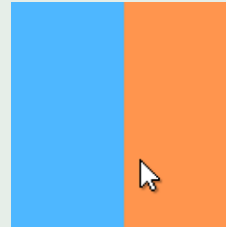
Color Wheel

Using the HSB color model, create a program that displays every hue on the screen in the form of a color wheel.



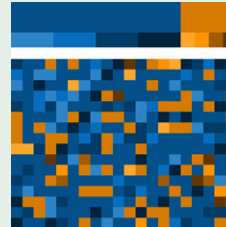
Threshold of Perception

Create a composition showing the smallest interval between colors that you can distinguish. One possible composition is a filled circle in front of a flat background, such that the colors of the background and circle are almost imperceptibly different.



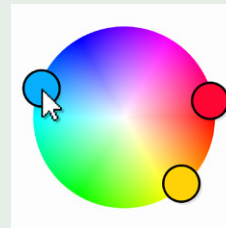
Interactive Complement

Complementary colors are 180° degrees apart on the color wheel. Split the canvas into two equal-sized rectangles, side by side. Using the HSB color model, create an interaction in which one rectangle’s color is controlled by the cursor position and the other rectangle contains the complement of this color.



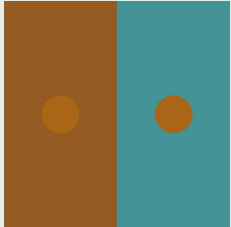
Accented Palette

Create a color scheme or palette with a dominant “base color” and an “accent color” formed from its complement. Devise a method of randomly sampling this palette such that the base color is selected roughly 75% of the time. Fill a grid of squares with colors selected using this method.



Split Complements

A color’s “split complements” are a pair of colors just adjacent ($\pm 15\text{--}30^\circ$) on the color wheel. Create an interactive sketch that displays swatches of the split complements for a color selected by the user.



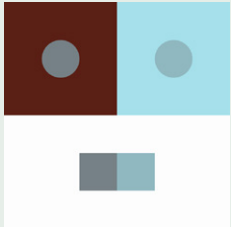
Josef Albers Color Relativity I

Study Josef Albers's color relativity exercises and write a program, like the one shown at left, that makes three colors look like four.¹ Control the components of the third color (in the spots) using your cursor. Under what conditions do the spots look the same or different?



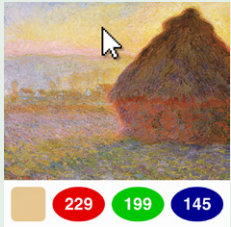
Palette from Photo

Select a photo. Write a program that derives an optimal five-color palette from your chosen image. (There are many ways to do this, but one of the most effective is k-means clustering.) If you can, use your palette to generate "separations" of your original image, like for silkscreening or risograph printing.



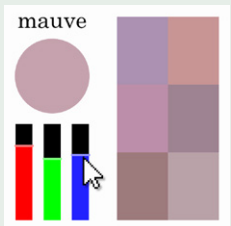
Josef Albers Color Relativity II

Using similar techniques as above, create a sketch in which four colors look like three. Demonstrate the relativity of color by duplicating the spots' colors in a location where they can be more easily compared. Can you write a program to generate novel color sets that always fulfill the four-look-like-three condition?



Color Inspector

Load and display a colorful image. Draw three ellipses to the screen and fill them with the red value, the green value, and the blue value, respectively, of an image pixel under the cursor.



Color Survey

Create an interface for specifying a color, using sliders to control its red, green, and blue components. Ask some friends to use your tool to create the colors mauve, teal, and plum. Save the data they create. In another sketch, load and display this data to compare their opinions.

Additional References

Taub Auerbach, *RGB Colorspace Atlas* (2011), <http://taubaauerbach.com/view.php?id=286>.

Carolyn L. Kane, *Chromatic Algorithms: Synthetic Color, Computer Art, and Aesthetics after Code* (Chicago: University of Chicago Press, 2014).

Rune Madsen, <http://printingcode.runemadsen.com/lecture-color/>.

Rune Madsen, <https://programmingdesignsystems.com/color/color-models-and-color-spaces/index.html>.

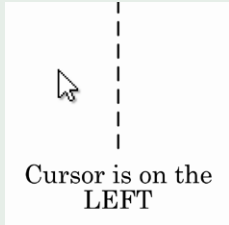
Rune Madsen, <https://programmingdesignsystems.com/color/perceptually-uniform-color-spaces/index.html>.

Robert Simmon, "Subtleties of Color," NASA Earth Observatory: Elegant Figures, last modified August 5, 2013, <https://earthobservatory.nasa.gov/blogs/elegantfigures/2013/08/05/subtleties-of-color-part-1-of-6/>.

Note

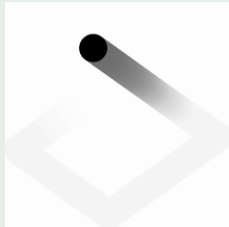
1. See Ticha Sethapakdi, *The Colorist Cookbook* (self-pub., 2015), <https://strangerbedfellows.files.wordpress.com/2015/12/the-colorist-cookbook.pdf>.

Conditional Testing



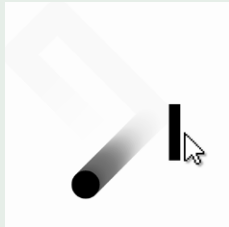
Left or Right

Create a sketch in which a text display indicates whether the cursor is on the left side or the right side of the canvas.



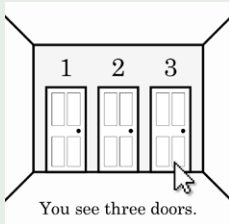
Billiard Ball

Create a sketch with a moving ball that bounces off the edges of the canvas. Take care that the ball never appears to overlap the edge of the canvas.



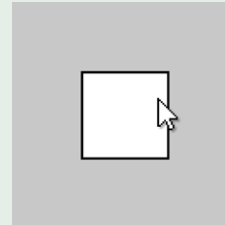
One-Person Pong

Create a sketch that recreates the game of Pong for a single user. Can you add a scoring system?



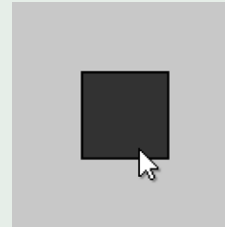
Choose Your Own Adventure

Create a branching narrative experience in which clicking in different regions of the screen leads to different rooms, situations, and outcomes. Enhance the experience with sound effects (e.g., door latches) and ambient audio recordings.



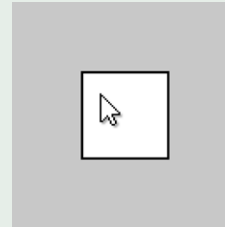
State Machine I

Place a white square on a gray background. Create an interaction wherein a click inside the square turns it black, after which it stays that way. (In this exercise and those that follow, make sure that clicks *outside* the square have no effect.)



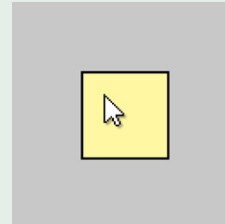
State Machine II

Place a white square on a gray background. Create an interaction wherein each click in the square flips its color. It should flip from white to black (if it is white) or from black to white (if it is black).



State Machine III

Place a white square on a gray background. Create an interaction wherein two clicks in the square are required to turn it from black to white, but three clicks are required to turn it from white to black.



State Machine IV

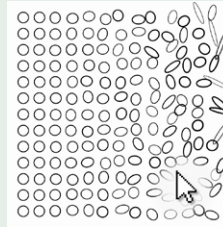
Place a white square on a gray background. Turn the square into a button with a "hover" state: make it white when inactive, yellow when the user is hovering over it (without having clicked), and black when the user is actively holding the mouse button down inside of it.

Unpredictability



Coin Toss

Create a sketch for a coin that is “tossed” every time the mouse button is clicked. Your coin should be evenly weighted so that heads and tails have the same likelihood of appearing. Test your program over ten tosses. What were the observed frequencies of heads and tails?



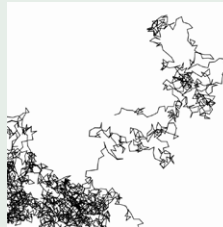
Order to Chaos

Make an interactive composition that depicts “order” when the cursor is on the left side of the canvas, and “chaos” when it is on the right. The amount of order or chaos (entropy) should vary smoothly according to the cursor position. The `map()` and `constrain()` functions may be helpful.



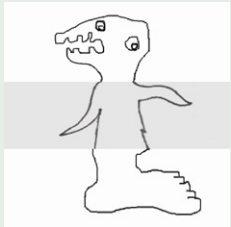
Roll the Dice

Create an app in which a virtual die is “rolled” every time the mouse is clicked. Your die should be evenly weighted, and each result should have an even 1-in-6 chance of appearing. Test your program over 18 clicks. How well does it perform? Modify your code so that it rolls six dice.



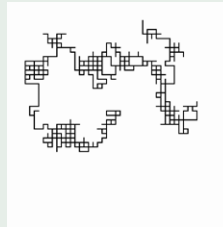
Drunk Walk I: Brownian Motion

Create a sketch in which a small 2D element travels erratically from one moment to the next, leaving a trail across the canvas as it moves. At every timestep, it should update its current position with a small random displacement in both X and Y.



Exquisite Corpse Machine

Ask some friends to prepare drawings of heads, torsos, and legs. Make sure these parts are able to line up interchangeably at the neck and waist. Create an “Exquisite Corpse” generator in which these parts are randomly recombined whenever a button is clicked. (Image: Tatyana Mustakos)



Drunk Walk II: Random Lattice Walk

Create a sketch in which a small element travels erratically from one moment to the next, leaving a trail as it moves. At every timestep, it should have a 1-in-4 chance of moving up, down, left, or right from its previous position.



Intermittent Events

Create a time-based sketch in which a brief event (such as a sound or animation) is triggered sporadically and unpredictably, with a very low probability. You may also display a scrolling timeline showing a history of when the event occurred.



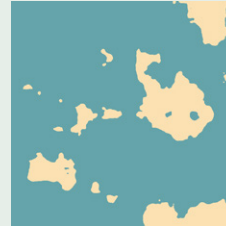
Drunk Walk III: Smoothed Noise

The smoothed quality of Perlin noise allows you to make an animation that is unpredictable, but also has temporal coherence. Use Perlin noise to dynamically regulate the size and position of a circle. Observe the effects of changing the parameters to the noise function.



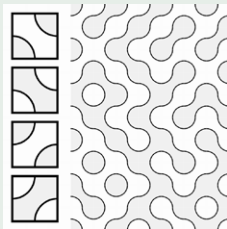
10 PRINT

In each cell of a square grid, choose with random probability to place a diagonal line pointing up or down. The fascinating maze-like patterns that arise are discussed at length in the book *10 PRINT CHR\$(205.5+RND(1)); : GOTO 10* by Nick Montfort et al.¹



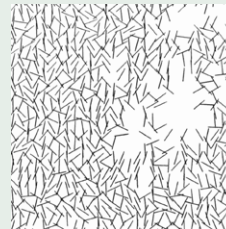
Imaginary Islands

Use a two-dimensional Perlin noise function to generate a map of imaginary islands. For every pixel: if the value of the noise function is below some threshold value, color the pixel blue (for water); if it is higher than the threshold, color it brown (for land).



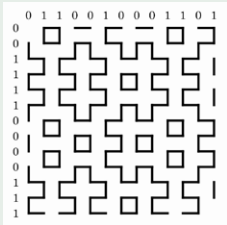
Duotone Truchet Tiling

Create a set of four duotone “Truchet tiles,” each with two quarter-circles connecting the midpoints of adjacent sides. Be sure to create all possible orientations and all possible colorings (corners light or corners dark). Randomly place these tiles in a square grid, enforcing continuity of color.



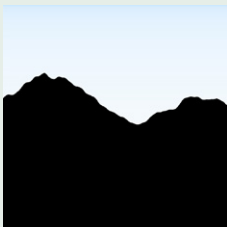
Recoding Molnár’s *Interruptions*

Carefully study some reproductions of the computational plotter work *Interruptions* (1968–1969) by Vera Molnár.³ Note how it consists of a grid of lines whose orientations have been randomized. Some lines are also missing, in patches. Write down ten additional observations about the work. To the best of your abilities, write a program to “re-code” *Interruptions*. Write a few sentences describing how your version succeeds or falls short.



Hitomezashi Sashiko Stitching

Randomly label the columns and rows of a grid with 0s and 1s. For each column, draw a vertical sequence of alternating dashes and gaps, beginning with a dash (for columns labeled 1) or a gap (for columns labeled 0). Similarly, draw horizontal sequences of dashes and gaps for the rows. Observe the resulting patterns.²



Noise Mountains

Generate some mountains by creating a series of parallel lines with no spacing between them, spanning from the bottom of the screen to an unpredictable height. Use a Perlin noise function to determine the height of each line.

Notes

1. *10 PRINT CHR\$(205.5+RND(1)); : GOTO 10*, ed. Nick Montfort et al. (Cambridge, MA: The MIT Press, 2012).

2. See Annie Perkins, tweet of March 29, 2020, https://twitter.com/anniek_p/status/1244220881347502080.

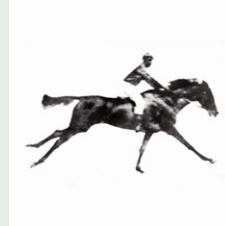
3. Vera Molnár, *Interruptions*, plotter artwork, 1968–1969, <http://dam.org/artists/phase-one/vera-molnar/artworks-bodies-of-work/works-from-the-1960s-70s>.

Arrays



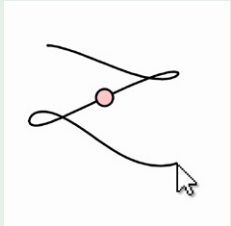
Living Line I

Create an interaction that stores the past 100 mouse positions and displays them as a polyline. Store the mouse data in three different ways: in two 1D arrays (one for X, one for Y); in one 2D array; and in an array of Point2D objects.



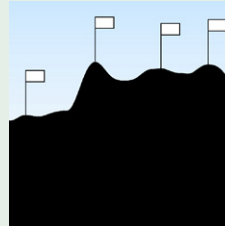
Animated Walk Cycle

Load each frame of an animated walk cycle into an array of images, and loop through these to display the character in motion. When a button is pressed, reverse the playback of the animation. (Image from Eadweard Muybridge, *Animals in Motion*, 1902.)



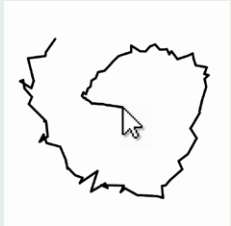
Living Line II

Create an interaction that stores the past 100 mouse positions and displays this path as a polyline. Move an animated ellipse along this path. When it reaches the end, return it to the start and repeat, or have it travel in the other direction.



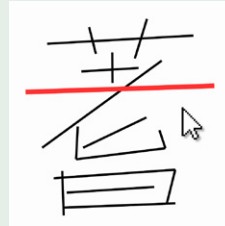
Plant the Flag

A bumpy “landscape” or terrain, consisting of an array of height values, has been provided for you. Write code that searches through this array and draws “flags” on the peaks (i.e., local maxima).



Living Line III

Create a line using the past 100 mouse positions, as above. Bring your line to life by progressively adding some randomness to each point.



Longest Line Search

Write a program that draws straight lines when the user clicks and drags. Color the longest line red.



Calligraphic Polyline

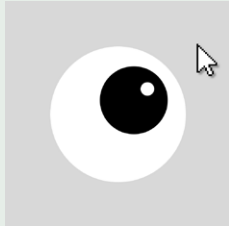
Create a sketch that stores the past 100 mouse positions. Draw a line between each point and the one previous to it. Make the thickness of this line inversely proportional to the distance between each pair of points, so that faster marks make thinner lines.



Reordering Rectangles

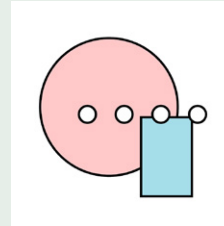
Create a program that stores the coordinates for some rectangles in an array. Reading from this array, your program should paint the rectangles over each other as they are rendered. Write code to reverse the array. Sort the rectangles from right to left. Sort the rectangles by their area.

Time and Interactivity



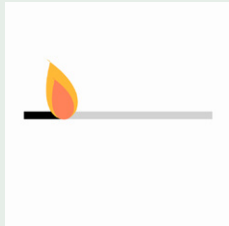
Eyes Following Cursor

Draw one or more eyes with pupils that follow the cursor. If you can, constrain each pupil to stay within its eyeball.



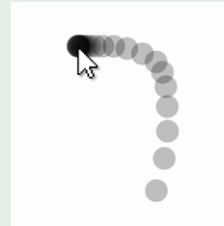
Abstract Typewriter

Create an expressive keyboard-based performance instrument. Each key should trigger a different animation, image, or sound. Carefully consider the aesthetics of the experience of playing your system. For inspiration, try *Patatap*, an audiovisual keyboard app by Jono Brandel.



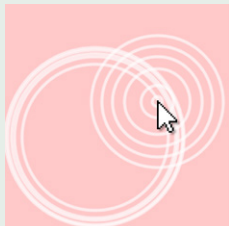
Fuse (or Progress Bar)

Program a virtual fuse that takes precisely five seconds to burn. Trigger an interesting event when it finishes, such as fireworks or a volcanic eruption. Consider alternate “skins” to tell a different story with the code for your fuse: a progress bar, a balloon inflating (and bursting).



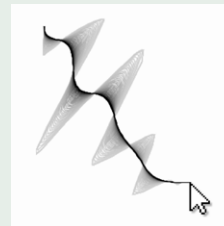
Easing: Filtering a Variable

Objects rarely move at a constant speed in the real world. Create a sketch in which an ellipse follows the position of the cursor but decelerates on approach. This requires an easing function or a filter for the mouse position values.



Ripples in a Pond

Create a program in which animated circular “ripples” emanate outward from the cursor each time the mouse button is clicked. Consider the speed at which your ripples expand. Implement your ripples in an object-oriented coding style.



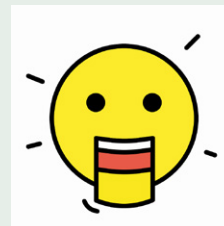
Smoothing Data

Create a polyline by storing and displaying a sequence of cursor positions. Now smooth it by progressively averaging each point along the line with its neighbors. Give consideration to how you handle the line’s terminal points.



Rain Catcher

Using code, animate a rainy day. Raindrops should appear from beyond the top of your screen and fall at random intervals. (It may be helpful to create a raindrop class.) Create a simple game in which the user can “catch” raindrops that fall close to the cursor.



Audio-Sensitive Animation

Create graphics that respond to the amplitude of microphone sounds. For example, you might illustrate the sound with a sleeping animal that is awakened by loud noises, or a face that appears to be voicing the microphone sound. Audio waveforms should be a starting point, not an end.

Typography

**STEAL
THIS
BOOK**

Ransom Letter

Create a sketch to display a brief text in which each letter is individually set with a random typeface and font size.



Scrolling Headlines (Chyron)

Choose a series of recent news headlines and write a program that displays them scrolling along the bottom of the screen. For an additional challenge, fetch the latest headlines automatically using a news API.

1uv2TYPE!

One-Line Typewriter

Create a sketch that accumulates and displays a string of letters as they are typed. For an additional challenge, delete characters when the backspace key is pressed.



Split-Flap Display (Word Ladder)

Curate a list of words that all have the same number of letters. Create a display that interpolates from one word to the next, in the manner of a split-flap airport sign, by iterating through the intermediate letters in alphabetical order.

GROW

Dynamic Text

Choose a word describing a bodily action, such as “grow,” “shiver,” or “jump.” Animate this word on your screen in a way that represents its meaning. For example, the word “grow” could become larger over time.

This is not a small voice
you hear this is a large
voice coming out of these cities.
This is the voice of LaTanya.
Kadesha. Shaniqua. This
is the voice of Antoine.
Darryl. Shaquille.

Word Finder

Typeset a brief text, such as a poem or email. Create a program that locates instances of a “word of interest” in that text. Your program should perform a visual treatment of that word, such as highlighting, underlining, or redaction.²

avoid

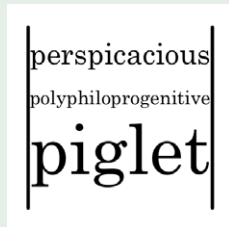
Responsive Text

Assign an interactive behavior to a word, such as “avoid” or “tickle,” that expressively defines its personality relative to the cursor.¹



Letterform Collage Tool

Create a tool that allows a user to make abstract collages or concrete poetry with letterforms. The user should be able to select a letter with the keyboard and then place it using the mouse. Save three compositions made with your tool.



Procrustean Typography

In Greek mythology, Procrustes stretched or cut different people to make them all fit in a certain iron bed. You will do the same with type. Create a program that captures words typed by the user and adjusts their font size so that they always fit the precise width of the canvas.



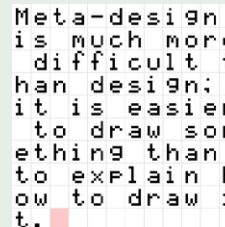
Text along a Curve

Set text along a curve, writing code to position each letter. If you can, orient each letter so that it is locally perpendicular to the curve.³



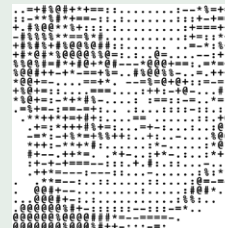
Glyph Hacking: Playing with Outlines

In this exercise you will load a typeface, modify its characters, and save out a new font of your own. Identify a method for loading font outlines into your preferred programming environment, such as by converting a TTF typeface to SVG using FontForge, or by accessing glyph contours directly with `textToPoints()` (in p5.js), opentype.js, the Rune.Font plugin⁴ (in Rune.js), or the Geomerative library⁵ (in Java Processing). Using code, create a treatment for all of the letters in the alphabet, such as making them puffy or spiky. Name your new font, and use your font to typeset its name. If you can, export your new glyphs to .ttf or another font format when you have finished.



Tiny Word Processor

Using a fixed-width font, create a word processor that allows a user to type characters into a field of cells. Provide a text cursor whose position can be moved with the arrow keys and mouse. Implement word-wrapping, deleting (with the backspace key), and text file exporting.



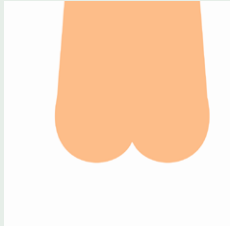
ASCII Vision

Write a program to generate an ASCII art representation of an image. To do this, you'll need a sequence of characters that represents different brightness values in an image. A common character ramp for representing 10 levels of grey is `"@%#*+=-.: "6`

Notes

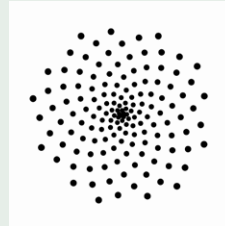
1. Adapted from Casey Reas and Ben Fry, "Typography," <https://processing.org/tutorials/typography/>.
2. The text in the poem is from Sonia Sanchez, "This Is Not a Small Voice" (1995), <https://poets.org/poem/not-small-voice>.
3. Daniel Shiffman, "Strings and Drawing Text," <https://processing.org/tutorials/text/>.
4. Rune Madsen, "Typography," <http://printingcode.runemadsen.com/lecture-typography/>.
5. Ricard Marxer, "Geomerative Library," <http://www.ricardmarxer.com/geomerative/>.
6. Paul Bourke, "Character Representation of Greyscale Images," <http://paulbourke.net/dataformats/asciart/>.

Curves



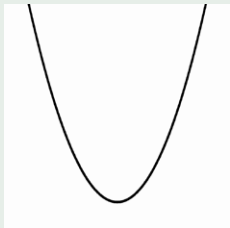
Butt Generator

Write a program that uses arcs or Bézier curves to generate images of one or more butts.¹



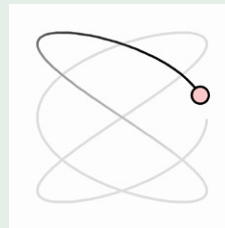
Phyllotaxis

Use turtle graphics to generate a phyllotactic spiral.³ Start your turtle at the center of the canvas. At each step, your turtle should draw a small element; move outwards by a slowly increasing amount; and rotate its orientation by the “Golden Angle,” ~137.507764°.



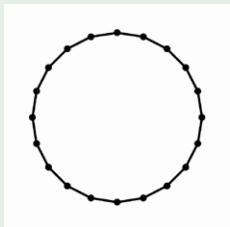
Parabola

A *parabola* is a curve whose formula is $y=ax^2$, where the variable a is some constant of your choice. Write a program that plots a parabola.



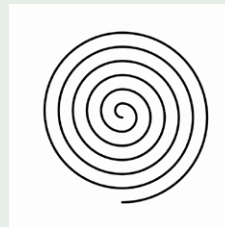
Lissajous

Lissajous curves are useful parametric functions of the form $x=\cos(a*t)$; $y=\sin(b*t)$, where a and b are typically small whole numbers. Use the mathematics of Lissajous curves to plot the movements of an animated element.



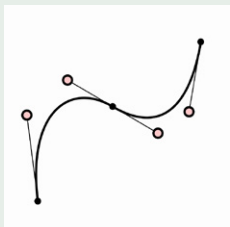
One Circle, Three Ways

Plot a circle (from scratch) three different ways: use the trigonometric functions $\sin()$ and $\cos()$ to plot a series of points; approximate a circle with four Bézier curves; and construct a circle with “turtle graphics,” using a series of alternating forward steps and small rotations.²



Spiral

Write a program that draws a spiral. Before you begin, research different types of spirals, such as Archimedes’s spiral (the radius of which grows arithmetically) and the logarithmic or equiangular spiral (whose radius grows geometrically). Consider different implementations, such as explicitly plotting your spiral using polar equations, implicitly rendering it by summing small differences (e.g., go forward, turn slightly, repeat), or approximating it piecewise with circular arcs.



Continuity of Bézier Curves

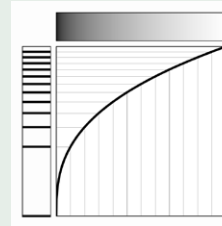
Create a curve by joining two Bézier curves. Now ensure that the combined curve is “C2 continuous,” with no visual discontinuities at the point where its components are joined. In other words, there should be continuity of position, tangent slope, and curvature.

Epitrochoid



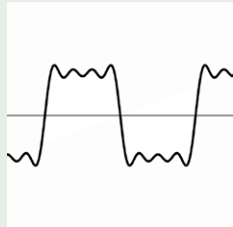
Polar Curve

Write a program to display a cardioid, epicycloid, hippopede, or other polar curve⁴ whose equation takes the form $r = f(\theta)$. The identities $x = r \cos(\theta)$ and $y = r \sin(\theta)$ may be helpful. Use the cursor to control parameters of the curve in real time.



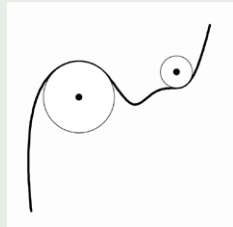
Shaping Functions

“Shaping functions” are indispensable in generating and altering signals for aesthetic purposes. Also known as tweens, interpolation curves, bias functions, easing curves, or unit generators, these functions are typically designed to receive and produce values between 0 and 1. Use a collection of shaping functions (such as p5.func by Luke DuBois, or Robert Penner’s Easing Functions) to create nonlinear relationships between time and position (to make animations) or between position and color (to make interesting gradients).⁶



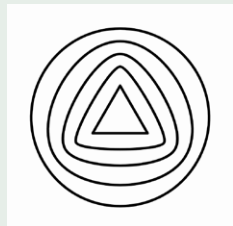
Fourier Synthesis

In Fourier synthesis, complex waveforms are made by summing up sine waves of different amplitudes and frequencies. For example, a square wave can be approximated by the sum: $\sin(x) + \sin(3x)/3 + \sin(5x)/5 + \sin(7x)/7 + \dots$. Generate a square wave with this technique.



Osculating Circle

Every point on a curve has a local *radius of curvature*: the radius of the osculating circle that best approximates the curve at that point. Write a program that displays the osculating circle for points along a mark drawn by the user. You can approximate this by computing a circle from three consecutive points along the mark.



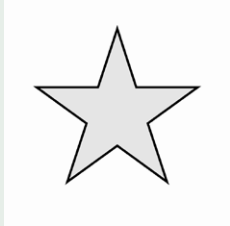
Circle Morphing

Create a program that interpolates between a circle and a triangle.⁵

Notes

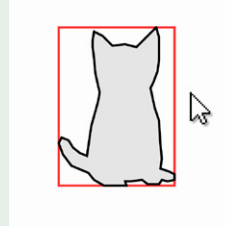
1. Inspired by Le Wei, “Butt Generator,” accessed April 11, 2020, <http://www.buttgenerator.com/>.
2. Yuki Yoshida, “Drawing a Circle Code Repository,” accessed April 11, 2020, <https://github.com/yukiy/drawCircle>.
3. Golan Levin, “Turtle’s Phyllotactic Spiral,” accessed April 11, 2020, <http://cmuems.com/2015c/deliverables/deliverables-09/#spiral>.
4. Mathworld, “Curves,” accessed April 11, 2020, <http://mathworld.wolfram.com/topics/Curves.html>.
5. See Dan Shiffman, “Guest Tutorial #7: Circle Morphing with Golan Levin,” *The Coding Train*, October 25, 2017, video, <https://www.youtube.com/watch?v=mvgcNOX8JGQ>.
6. Luke DuBois, “p5.func,” accessed July 27, 2020, <https://idmnyu.github.io/p5.js-func/>; Robert Penner, “Easing Functions,” accessed July 27, 2020, <http://robertpenner.com/easing/>.

Shapes



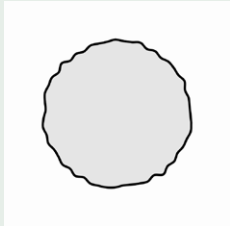
Make a Star

Plot the vertices of a regular 10-sided polygon, using the trigonometric functions `sin()` and `cos()` in the same way that you might generate a circle. Modify your code to generate a star by alternating long and short radii.¹



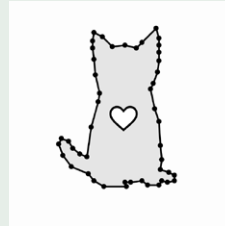
Axis-Aligned Minimum Bounding Box

Given a 2D shape represented by a set of points, compute and plot its “bounding box”: a useful rectangle defined by the leftmost, rightmost, highest, and lowest points on the shape’s contour. Create an interaction in which the shape is displayed differently if the cursor enters its bounding box.



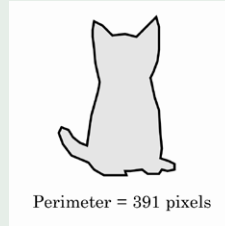
Random Splat

Plot the vertices of a many-sided polygon, using the functions `sin()` and `cos()` in the same way that you might generate a circle, but compute the position of each point with a radius that is slightly randomized. How closely can you make the result resemble a drop of ink?²



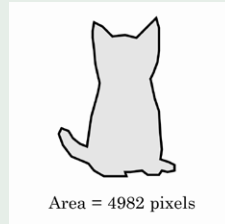
Connect the Dots

The numbers below define the vertices of a polygon. Write code that connects the dots to plot the shape. `x={81, 83, 83, 83, 83, 82, 79, 77, 80, 83, 84, 85, 84, 90, 94, 94, 89, 85, 83, 75, 71, 63, 59, 60, 44, 37, 33, 21, 15, 12, 14, 19, 22, 27, 32, 35, 40, 41, 38, 37, 36, 36, 37, 43, 50, 59, 67, 71}; y={10, 17, 22, 27, 33, 41, 49, 53, 67, 76, 93, 103, 110, 112, 114, 118, 119, 118, 121, 121, 118, 119, 119, 122, 122, 118, 113, 108, 100, 92, 88, 90, 95, 99, 101, 80, 62, 56, 43, 32, 24, 19, 13, 16, 23, 22, 24, 20};`



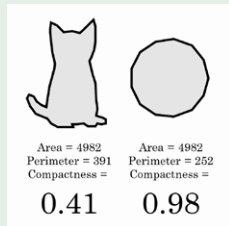
Computing the Perimeter

Given a 2D shape represented as a set of points, calculate the length (or *perimeter*) of its outline. To do this, add up the distances between every pair of consecutive points along its boundary. Don’t forget to include the distance from the last point back to the first one.



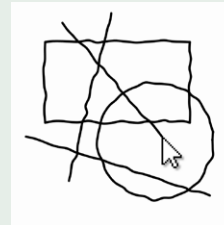
Computing the Area

Given a simple (non-self-intersecting) 2D shape whose points are stored in arrays `x[]` and `y[]`, calculate its area. This can be done using Gauss’s area formula, or “shoelace algorithm,” which sums, for all points `i`, the products $((x[i+1] + x[i]) * (y[i+1] - y[i]))/2.0$.



Shape Metrics: Compactness

One of the simplest shape metrics is “compactness” (or isoperimetric quotient), which describes the extent to which a shape does or doesn’t sprawl. Often used in redistricting to avoid gerrymandering, compactness is a dimensionless, scale-invariant, and rotation-invariant quantity.³ It is computed by taking the ratio of a shape’s area to its squared perimeter. Compute the compactness of the provided shape. Compare this value with the compactness of some other shapes.

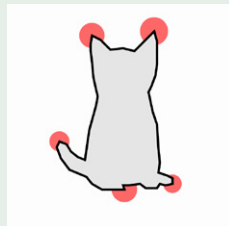
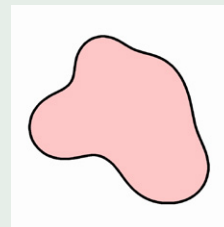


Hand-Drawn Graphics Library

Create a set of functions for rendering basic shapes with a hand-drawn feel. At a minimum, your library should provide functions for drawing line segments, ellipses, and rectangles.

Blob

Using any means you prefer, design and animate an expressive 2D blob. For example: you could create a closed loop of Bézier curves; trace the contours of metaballs (implicit curves) using Marching Squares; calculate a Cassini ellipse, cranioid, or other parametric curve; or simulate a smooth ropelike contour using Verlet integration.



Detecting Points of High Curvature

Given a 2D shape represented as a set of points, we can estimate the “local curvature” along the shape’s boundary by computing the angle between consecutive trios of points. Using the provided shape, write code to determine which points have especially high curvature (i.e. the fingertips) and mark them with a colored dot. *Hint:* Use the dot product to compute the angles, and the cross product to distinguish positive curvature (convexities) from negative curvature (concavities).

Notes

1. Adapted from Rune Madsen, “Shape: Procedural Shapes,” Programming Design Systems, accessed July 6, 2020, <https://programmingdesignsystems.com/shape/procedural-shapes/index.html>.

2. Adapted from Madsen, “Shape: Procedural Shapes.”

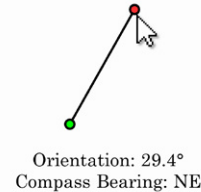
3. Wikipedia, s.v. “Compactness measure of a shape,” last modified June 22, 2020, https://en.wikipedia.org/wiki/Compactness_measure_of_a_shape.

Geometry



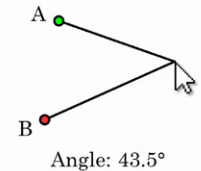
Midpoint of a Line Segment

Write a program that draws two random points (A and B) whenever a button is pressed. Connect these points with a line segment. Calculate the midpoint of this line segment, and place a dot there. Place an additional circle one-third of the way from A to B.



Compass Orientation

Store the locations of the two most recent mouse clicks and draw a line between them. Using the `atan2()` function, compute the angular orientation of this line. Display this angle in degrees, and label it with the nearest compass direction (N, NE, E, SE, S, SW, W, NW).



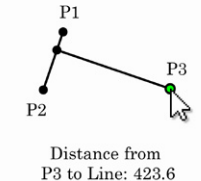
Angle between Three Points

Write a program that computes the angle between three points: two randomly placed points, A and B, and the mouse cursor, C. *Hint:* Use the dot product to compute the angles, and use the cross product to distinguish positive from negative curvature.



Construction of a Perpendicular

Write a program that draws a line from the center of the canvas to the cursor. Construct a second line that starts at the cursor, is 50 pixels long, and is perpendicular to the first line.



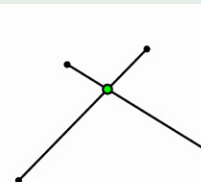
Distance from a Point to a Line

Write a program that creates a random line segment whenever a key is pressed. Compute and display the shortest distance between this line and the cursor. Place a dot at the point on this line that is closest to the cursor.¹



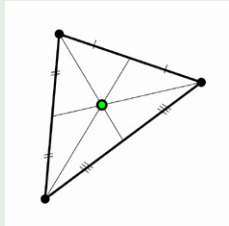
Parallel Polyline (Offset Curve)

Write a program that stores cursor points as a user draws. Connect these points with a polyline. Use geometry to calculate another polyline, which is offset everywhere from the user's drawing by a distance of 50 pixels.



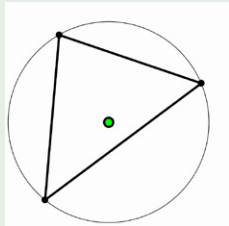
Intersection of Two Line Segments

Write a program that creates two random line segments whenever a button is pressed. Calculate the intersection point of these two line segments, and if it exists, draw a dot there.²



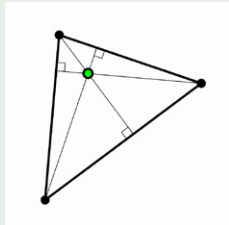
Centroid of a Triangle

Construct a triangle from three random points whenever a button is pressed. Draw lines (called “medians”) to connect the midpoints of each side to its opposite vertex. At the intersection of these medians is the triangle’s *centroid*. Place a dot there.



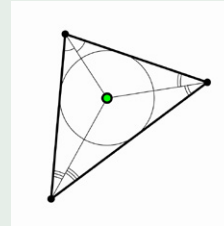
Circle from Three Points (Circumcenter)

Construct a random triangle whenever a button is pressed. Generate a circle that passes precisely through all three of its vertices. The center of this circle is called the *circumcenter* of the triangle. Place a dot there. Note: the circumcenter is not always inside the triangle.³



Orthocenter of a Triangle

The “altitude” of a triangle is a line that is perpendicular to a given side and passes through the opposite vertex. Write a program that constructs a triangle from three random points whenever a button is pressed. Calculate and display the triangle’s three altitudes. The *orthocenter* of this triangle is located at the intersection of its altitudes. Place a dot there. Note: for most triangles, the altitudes will not pass through the midpoints of the sides.



Incenter of a Triangle

The *incenter* of a triangle is located at the intersection of the angle bisectors of a triangle’s three corners. Write a program that generates a random triangle whenever a button is pressed and places a dot at its incenter. (The incenter also happens to be the center of a circle inscribed inside the triangle, called the incircle; its radius is obtained by dropping a perpendicular line from the incenter to any of the triangle’s sides. If you can, draw the triangle’s incircle.)

Notes

1. Paul Bourke, “Points, Lines, and Planes,” 1988–2013, <http://paulbourke.net/geometry/pointlineplane/>.
2. Bourke, “Points, Lines, and Planes.”
3. Paul Bourke, “Equation of a Circle from 3 Points (2 Dimensions),” January 1990, <http://paulbourke.net/geometry/circlesphere/>.

Image



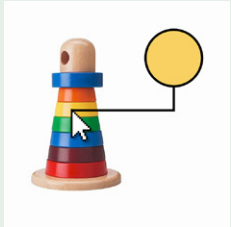
Collage Machine

Collect a directory of images. Write a program that uses these to generate collages of images. Include some unpredictability within it so that it generates a different collage each time it runs.



Searching for the Brightest Point

Display the live video feed from your webcam to the screen and process the pixels to find the brightest point in the image. Place an indicator at the location of this pixel.



Color of a Pixel

Display an image. Create an interactive system that continually fetches the color of the pixel under the mouse as it passes over this image. Use this color to fill a shape drawn to the screen as the mouse is moved around.



Image Averaging

Collect ten or more images labeled with the same word or concept, such as "apple," "sunset," or "clock." Ensure that all of the images have the same pixel dimensions. Compute a new image that is the pixelwise average of the collection, as a way to visualize regularities in the representation of that concept.



Subsample and Downsample

Write a program that pixelates an image to produce a low-resolution version. Begin by *subsampling* the image (destination pixels are selected from the original). Then, *downsample* the image (destination pixels are local averages).



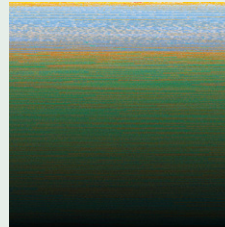
Edge Detector (Sobel Filter)

Write a program that uses a Sobel filter to detect and display the edges in an image. If you can, compute the strength and orientation of these edges.



Random Dot Dithering

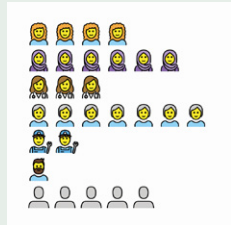
Load an image into memory and fetch the brightness of a pixel in a random location. Generate a random number between 0 and 255; if it is greater than the brightness of the pixel, draw a black dot at a corresponding location on the canvas. Repeat as the picture emerges.



Pixel Sort

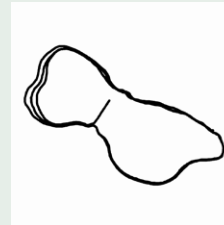
Sort the pixels of an image by brightness. Then repeat the exercise, but sort by hue. For tips, see Dan Shiffman's "Coding Challenge #47: Pixel Sorting in Processing" in his *Coding Train* video of December 21, 2016.

Visualization



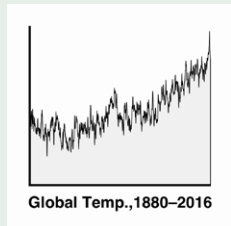
Text Message Isotype

Collect the text messages that you sent and received over the past week. Count and categorize these messages according to person, topic, or mood. Create an isotype visualization to represent this data pictorially.¹



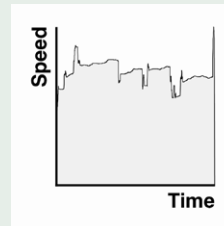
Path Plotting I

Using a smartphone app such as OpenPaths or SensorLog, record your GPS location data for a duration of at least one week. Export the latitude and longitude data to a CSV file. Write code to load and visualize the paths of your daily journeys. What observations can you make?



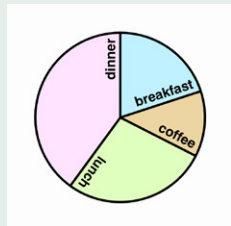
Temperature Timeline

Download a dataset of the average monthly temperatures where you live. Visualize this data in a chart showing time on the x-axis and temperature on the y-axis. Make three versions of the graph, showing the data as points, as vertical bars, and as data points connected with a curved line.²



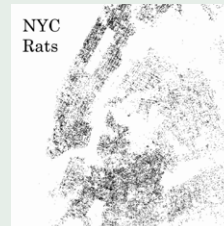
Path Plotting II

Reuse the location dataset from the previous exercise and visualize it in a way that is *not* a map. Consider displaying information such as path lengths or speed, frequency of visits to a particular location, distance traveled per day, etc.



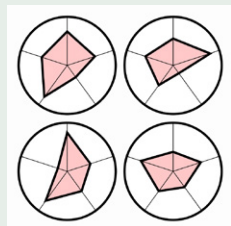
Pie Chart

Write code to generate a pie chart that displays what you spent on breakfast, lunch, dinner, and coffee yesterday. You will need to use a function that renders a filled arc. Load the data from an external file in a well-structured format like JSON or CSV. Make sure your chart has a key.



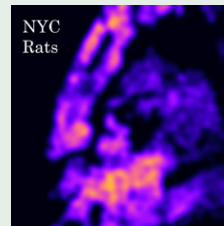
Dot Map

The “Rat Sightings” dataset from NYC OpenData lists all of the complaints about rats made to New York City’s 311 response center since 2010. Write a program to map this geolocated data by representing each complaint as a single dot.⁴



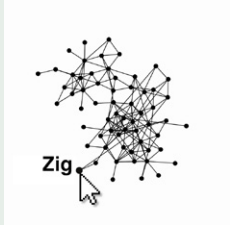
Small Multiple of Radar Charts

Take a “Big Five” personality quiz to (ostensibly) quantify your conscientiousness, agreeableness, neuroticism, openness, and extraversion.³ Generate a radar chart to visualize this multivariate data. Ask some friends to take the quiz, and create a trellis plot (or “small multiple”) that presents their personality charts side by side.



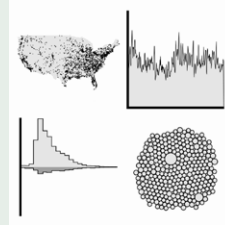
Heat Map

Write a program to display the Rat Sightings data as a spatial heat map. This presents two challenges, as you will need to: design a density function that represents the density of dots in a map as a continuous field, and select a color palette that represents the intensity of this field.



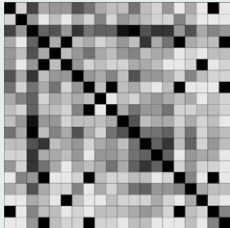
Social Network Graph

Develop a “force directed graph” to visualize a social network, in which individuals are represented as nodes and their relations as links, and the layout is governed by a simulated particle system. Use this to visualize the community of 62 bottlenose dolphins living off Doubtful Sound, New Zealand, as compiled by Lusseau et al.



One Dataset, Four Ways

Visualize the Slate USA Gun Deaths dataset (2013) in four different ways: a map, a timeline, a bar chart based on age, and a fourth method of your choice. If you can, provide an interaction that allows a user to zoom, sort, filter, or query the data. Write about how the different displays provide different insights.⁷



Social Network Matrix

A *symmetric adjacency matrix* visualizes social interactions in a group. Individuals are listed along a table’s rows and columns, and interactions between each pair are quantified in the cells. Create such a visualization for the family of wild monkeys observed by Linda Wolfe as they sported by a river in Ocala, Florida.



Real-Time Data Display

Locate a real-time Internet data stream such as that published by a sensor or an API (weather, stock prices, satellite locations). Create a program that displays this data in an evocative way. The example here shows the location of the International Space Station, available from open-notify.org.⁵



Web Scraping

Write a program to exhaustively “scrape” a collection of online images. For example, you might automate a process to download images of every product offered by a certain vendor. Write a second program to display your images in a grid. How else could you organize the collection?⁶

Notes

1. Otto Neurath, *International Picture Language: The First Rules of Isotype* (London: K. Paul, Trench, Trubner & Company, Limited, 1936).
2. Ben Fry, “Time Series,” *Visualizing Data: Exploring and Explaining Data with the Processing Environment*, chap. 4 (Cambridge, MA: O’Reilly Media, Inc., 2008).
3. For example: Open-Source Psychometrics Project, “Big Five Personality Test,” <https://openpsychometrics.org/tests/IPIP-BFFM/>.
4. The Rat Sightings dataset is available online at <https://data.cityofnewyork.us/Social-Services/Rat-Sightings/3q43-55fe>. See also Fry, *Visualizing Data*, chap. 6.
5. OpenNotify.org, International Space Station Current Location, <http://open-notify.org/Open-Notify-API/ISS-Location-Now/>. Also see Dan Shiffman’s sequence of video tutorials, *Working with Data and APIs in JavaScript*, last updated on July 8, 2019, https://www.youtube.com/playlist?list=PLRqwx-V7Uu6YxDKpFzf_2D84p0cyk4T7X.
6. See Sam Lavigne’s sequence of tutorials, *Scrapism*, last updated on May 27, 2020, <https://scrapism.lav.io/>.
7. Dataset available at http://www.slate.com/articles/news_and_politics/crime/2012/12/gun_death_tally_every_american_gun_death_since_newtown_sandy_hook_shooting.html.

Text and Language

I typed BLUE!



String Search

Find a table of color names and their corresponding RGB values. Create an interaction in which a square becomes colored when a user types a known color name. (Is your program case-insensitive?)

Given their physiological requirements, limited dispersal abilities, and hydrologically sensitive habitats, amphibians are likely to be highly sensitive to future climactic changes.

7.09

Frog and Toad ate many cookies, one after another. "You know, Toad," said Frog, with his mouth full, "I think we should stop eating. We will soon be sick." "You are right," said Toad.

3.94

Average Word Length

Write a program that calculates the average word length of a provided text. This is a useful approximation of a text's "reading level." Run your program on several different source materials.

de·klept·ism
un·bibl·ing
dys·clar·tion
re·bio·meter
rhino·rupt·ed

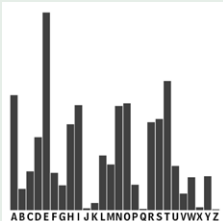
Nonsense Words

Find some lists of common English prefixes, word roots, and suffixes. Select a random item from each list and combine them in a simple syntax (prefix+root+suffix) to generate plausible nonsense words. What might these words mean?

a act all and and
and and another
are are beings
born brotherhood
conscience dignity
endowed equal free
human in in of one
reason rights
should spirit they
towards with

Sorting Words

Load a document and display its words (a) sorted alphabetically, (b) sorted by their length, and (c) sorted according to their frequency in the text.



Letter Frequency

Write a program to calculate the frequencies of the letters in a provided text. (Be careful to make your program "case-insensitive.") Write code to generate a visualization (such as a bar chart or pie chart) of the letters' frequencies.

WNYC Employees Demanded Diversity. They Got Another Pandemic to Cut Service That Macron Replaces France's Prime Marijuana Scholar, Dies at 92 After Fighting Plastic Waste Minister in Bid For Fresh Start Won't Return

Cut-Up Machine

In the *Dada Manifesto*, Tristan Tzara describes using a newspaper, scissors, and some gentle shaking to generate irrational poetry. Do the same with code. Write a program that randomizes the lines or sentences of a newspaper article to make a Dada-style poem.

	A	B	C	D	E	F	G	H	I
A	0.00	0.12	0.44	0.20	0.83	0.23	0.34	3.00	0.17
B	0.19	0.01	0.00	0.00	0.03	0.00	0.00	0.01	0.05
C	0.12	0.00	0.04	0.00	0.26	0.00	0.00	0.00	0.50
D	0.39	0.00	0.00	0.04	3.28	0.00	0.00	0.00	0.26
E	0.00	0.49	0.52	0.73	0.50	0.24	0.38	1.79	0.30
F	0.06	0.00	0.00	0.00	0.15	0.09	0.00	0.01	0.24
G	0.18	0.00	0.00	0.03	0.26	0.00	0.04	0.00	0.26
H	0.01	0.00	0.61	0.00	0.02	0.00	0.34	0.00	0.00
I	0.17	0.09	0.20	0.33	0.22	0.25	0.36	3.00	0.01

Letter-Pair Frequency

Write a program to calculate the frequencies of letter pairs (character 2-grams, such as "aa," "ab," "ac") in a large text source. Plot the frequencies in a 26x26 matrix.

35 not like
34 i do
34 do not
33 like them
29 in a
21 eat them
18 with a
18 not in
15 i will
14 i would
14 them in
13 would not
12 would you
11 eggs and

Bigram Calculator

Write a program to calculate the frequency of all bigrams (word-pairs) in a document. *Advanced students:* develop a program that judges the similarity of two text files based on how many bigrams they have in common.

Dammit, Jim!
I'm a **marriage therapist**, not a **meat packer**!

Dammit Jim

Find a list of occupations. Use these in a generative grammar that produces sentences in this format: "Dammit, Jim, I'm an X, not a Y!" (popularized by the sci-fi TV show *Star Trek*). Be sure to write "an X" if X begins with a vowel and "a X" if it begins with a consonant.¹

I do not like
green **junctions**
and **switch**.
I do not like
them, **Monorail**.

Noun Swizzler

Load a text, and replace each noun with a randomly selected noun taken from a second text. You may need to use a "part-of-speech tagger" to identify the nouns. In your substitutions, try to match the use of plural and proper nouns in the first text.

Knock, knock!
Who's there?
Child.
Child who?
A young person.

Knock-Knock Joke Generator

Create a program that generates knock-knock jokes. At a minimum, your program should select a random word as a response to "Who's there?" and add to this word to create the final line of the joke. Generate ten jokes.

It is hard to
erase blue or
red **ink**.
Dunk stale
biscuits into
strong **drink**.

Rhyming Couplets

Select and load a large expository text. Create a program that finds rhyming couplets within it, in order to make new poems. You may need to use an additional library (such as RiTa) to tell you what the words sound like.²

onceway uponway a
idnightmay earydray,
ilewhay i oenderpay,
eakway andyay
earyway, overway
anymay aaintquay
andyay uriouscay
olumevay ofyay
orgottenfay orelay

Pig Latin Translator

Create a program that translates provided text into Pig Latin. In this playful scheme, the initial consonant (or consonant cluster) of each word is transferred to the end of that word, after which the syllable "ay" is added.

Meg and Jo closed their
weary eyes, and lay at rest,
like storm beaten boats

Mrs. Brooke, with her
apron over her head, sat
sobbing dismally.

"Do you remember
our castles in the air?"
asked Amy, smiling

Haiku Finder

Write a program that automatically discovers "inadvertent haikus": sentences in a chosen text whose words happen to fall in groups of five, seven, and five syllables. A basic solution will discover haikus with awkward breaks; add heuristics to improve the quality of your results.

Fubour scubore uband
subevuben yubears
ubagubo ubour
fubathubers
brubought fuborth,
ubupubon thubis
cubontubinubent,
uba nubew nubatiubon,
cuboncubeived ubin
lubibubertuby

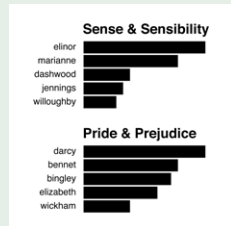
Argots and Language Games

Investigate argots, secret languages, and word games like Ubbi Dubbi, Tutnese, Pirate English, and Dizzouble Dizzutch. Select one and write a program that translates a provided text into it.

I do not eat like
them in like
green eat them
in they like them.
could you eat like
them and you?

Markov Text Generator

In a Markov chain, a dataset of letter-pair, bigram, or n-gram frequencies is used as a "probability transition matrix" to synthesize new text that statistically resembles the dataset's source. Build a Markov generator using the data you collected earlier.³



There once was
a bug who liked
art.

He thought it
was awfully
tart.

Keyword Extraction with TF-IDF

Obtain a collection of related documents, such as poems, recipes, or obituaries. Write a program that uses the TF-IDF ("Term Frequency - Inverse Document Frequency") algorithm to determine the keywords that best characterize each document.⁴

Limerick Generator

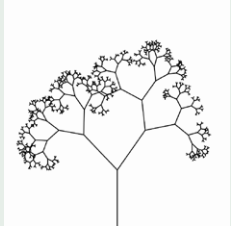
Limerick poems have five lines in an AABBA rhyming pattern. The building block of these lines is the *anapest*: a foot of verse consisting of three syllables, the third of which is accented: *da-da-DA*. Lines 1, 2, and 5 consist of three anapests; they end with a similar phoneme in order to create the rhyme. Lines 3 and 4 also rhyme with each other, but are shorter, consisting of two anapests each. Write a program to generate limericks. Use a code library such as RiTa to help evaluate your words' rhymes, syllables, and stress patterns.⁵

Notes

Exercises in this section were contributed by Allison Parrish.

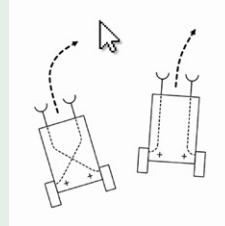
1. See Darius Kazemi's repository of structured corpora: <https://github.com/dariusk/corpora>. Also consider code libraries for producing grammar structures, like Tracery by Kate Compton, <http://www.crystalcodepalace.com/tracery.html>.
2. For Java and JavaScript, see RiTa by Daniel Howe, "RiTa, a Software Toolkit for Computational Literature," <https://rednoise.org/rita/>. For Python, see NLTK, "Natural Language Toolkit," <https://www.nltk.org/>.
3. For Python see: "Markovify," <https://github.com/jsvine/markovify>.
4. See Dan Shiffman, "Coding Challenge #40.3: TF-IDF," *The Coding Train*, October 12, 2016, video, <https://www.youtube.com/watch?v=RPMYV-eb6II>.
5. Howe, "RiTa"; see "The CMU Pronouncing Dictionary," <http://www.speech.cs.cmu.edu/cgi-bin/cmudict>.

Simulation



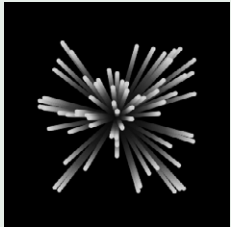
Recursive Tree

Use recursion to create a tree. Begin with a symmetrical design that repeatedly bifurcates. Introduce a variable that proportionally reduces the length of each iteration's branches, and another to alter their orientation. Explore your tree's possibilities by controlling these variables with the cursor.¹



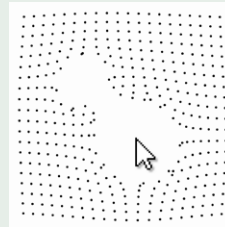
Braitenberg Vehicles

A Braitenberg vehicle is an autonomous agent that moves and steers based on sensor inputs. It measures a stimulus at its location; depending how this signal is mapped to each wheel's power, the vehicle can appear to exhibit different goals and behaviors. Implement vehicles that steer toward or away from the cursor.



Fireworks (Particle Shower)

Create a particle class that stores a speck's 2D position and velocity. Add methods to give the particle an initial, randomly generated velocity and constant acceleration. Create an array of particles to simulate a firework. Each element of the array will need to start from the same position.



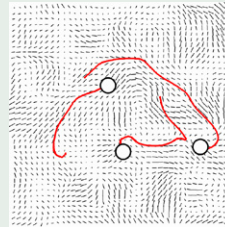
Cursor-Sensitive Particles

Create a particle class that stores a speck's 2D position and velocity. Add a method that enables a particle's motion to be affected by simulated forces. (See Euler integration and Newton's 2nd Law.) Create an array of particles that are attracted to or repelled by the cursor.



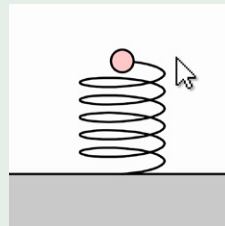
Flocking

Create a two-dimensional flock of creatures, modeling your animals as particles that exert (and are affected by) forces of mutual *separation* (to prevent collision), *cohesion* (to stick together as a group), and *alignment* (to orient toward similar directions as their neighbors). Connect your governing parameters to sliders or other UI controls, and observe how changing the relative strengths of these forces alters the behavior of the flock or swarm. Include other forces, such as the desire to flee from a predator, the hunger to hunt for food, etc. Be sure to represent your creatures with a graphic that makes their orientation visible.²



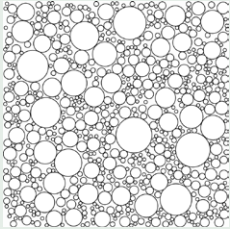
Flow Field

Use 2D Perlin noise to compute a flow field, such that every location on the canvas has an associated x-force and y-force. Place particles into this field, impelling them with the forces at their locations. Make recordings of their traces. Confine the particles with periodic boundaries.



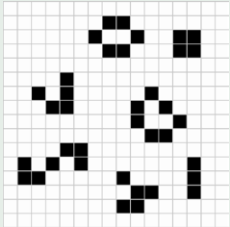
Spring

Model a bouncy spring as a particle with a current position **P**, velocity **V**, and rest position **R**. When moved from **R**, a restorative force **F** proportional to this displacement pushes it back. Update the particle using Euler integration: add **F** into **V**, reduce **V** by a percentage due to damping, and add **V** into **P**.



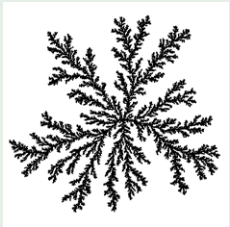
Circle Packing

Generate a “circle packing”: an arrangement of circles such that none overlap, and some (or all) are mutually tangent. In one approach, randomly placed circles are added to unclaimed regions of the canvas and grow until they collide with any previous circles.³



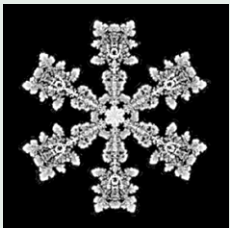
Conway's Game of Life

Create an implementation of “Conway's Game of Life.” This classic cellular automaton is one of the simplest and best illustrations of the way in which complex patterns of self-organization can emerge from simple rules.⁴



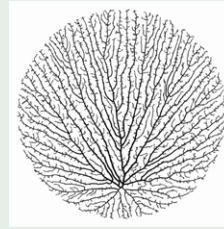
Diffusion-Limited Aggregation

Implement and explore the coral forms that arise from diffusion-limited aggregation (DLA): a simulation in which meandering particles become fixed in place when they collide with previously fixed particles (or an initial “seed”).⁵



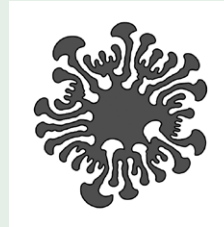
Snowflake Generator

Snowflakes are thought to form through a DLA-like process. To make a snowflake generator, modify your DLA simulation from the previous exercise so that it has dihexagonal symmetry.



Space Colonization

Implement *space colonization*: an iterative algorithm, first described by Adam Runions, for growing networks of branching line structures based on the locations of “growth hormone” sources to which the lines are attracted.⁶



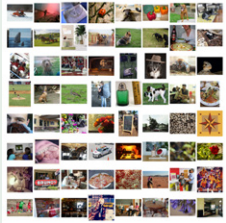
Differential Growth

Implement *differential growth*, in which a chain of connected nodes (represented by a curve or polyline) uses simple rules like attraction, repulsion, and alignment to produce meandering shapes. The algorithm uses *adaptive subdivision* to interpose new nodes when two adjacent nodes get too far apart.⁶

Notes

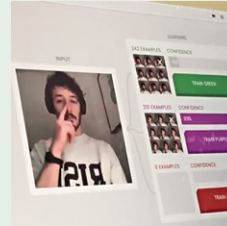
1. Dan Shiffman, “Coding Challenge #14: Fractal Trees - Recursive,” *The Coding Train*, May 30, 2016, video, 15:52, <https://www.youtube.com/watch?v=0jjeOYMjmDU>.
2. “6.1: Autonomous Agents and Steering - The Nature of Code,” *The Coding Train*, August 8, 2015, video, 14:28, <https://www.youtube.com/watch?v=Jlz2L4tn5kM>. Also see Craig Reynolds's research into steering behaviors and “boids.”
3. “Coding Challenge #50.1: Animated Circle Packing - Part 1,” *The Coding Train*, January 9, 2017, video, 28:31, <https://www.youtube.com/watch?v=QHEQuolKgNE>.
4. “7.3: The Game of Life - The Nature of Code,” *The Coding Train*, August 10, 2015, video, 16:03, <https://www.youtube.com/watch?v=tENSCE0-Lec>.
5. “Coding Challenge #34: Diffusion-Limited Aggregation,” *The Coding Train*, August 18, 2016, video, 47:06, https://www.youtube.com/watch?v=CI_Gjj80gPE.
6. Jason Webb, “Morphogenesis Resources,” 2020, <https://github.com/jasonwebb/morphogenesis-resources>.

Machine Learning



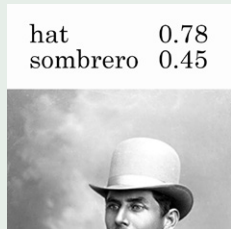
Dataset History Audit

Identify a widely used dataset, such as ImageNet, MNIST, or LFW. Research its content and genesis. Who made it, when, and how? Who uses it, and for what? (What biases might this dataset have?) Describe your findings in one or two paragraphs.



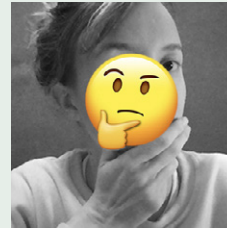
Don't Touch Your Face

Touching your face can spread disease. Train a webcam classifier to detect when you touch your face. Write a program that sounds an alarm if you do. (Image: Isaac Blankensmith's *ANTI-FACE-TOUCHING MACHINE™*, an influential implementation of this concept.)²



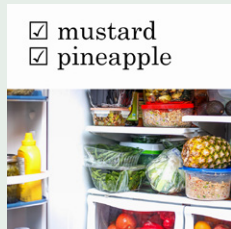
Comparing Models

Use two different image analysis or classification tools to interpret the same image. Write a paragraph that describes and compares their results.



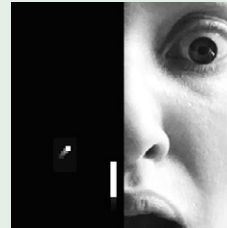
Emoji Translator

Using an image classifier and your computer's camera, train a system that detects your facial expressions and displays corresponding emojis.³



Shopping List

Use an object recognition library or a classifier to store a list of all the objects it sees. Use it to make a shopping list of everything in your fridge.¹



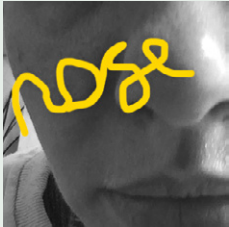
Body as Game Controller

Train an image classifier to determine whether you have raised your left or right hand. Using a "webdriver" (also called a mouse/keyboard automator), write a program in which your classifier controls a classic arcade game, such as Space Invaders, by spoofing presses of the WASD or arrow keys. Examples of webdrivers include the Java Robot class, JavascriptExecutor, and Selenium Browser Automation Project.



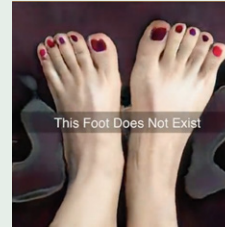
What Do You See?

Combine an object recognition classifier with a text-to-speech library. Write a computer program that narrates what it sees through the webcam.



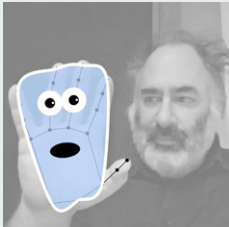
Draw with Your Nose

Use a pose classifier or face tracker to create a program that lets you draw with your nose.⁴



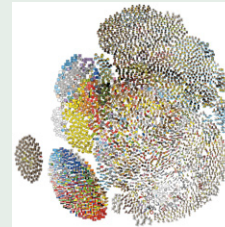
More Like This, Please

Create or download a collection of about a thousand images representing a narrow category of subject matter (cats, flowers, yearbook photos). Using a Generative Adversarial Network (GAN), synthesize new images that appear to belong to this dataset.⁷ (Image: “This Foot Does Not Exist” by the MSCHF collective.)



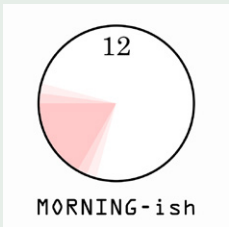
Hand Puppet

Train a webcam regressor to produce a number between zero and one, according to the closing and opening of your hand. (Your regressor could use hand pose data from a tracking library, or it could process the camera's pixels directly.)⁵ Use this number to puppeteer the mouth of a simple cartoon face.



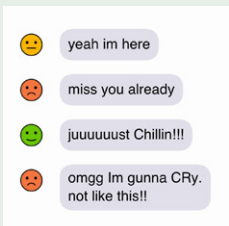
Clustering a Collection of Images

Generate a 2D map that reveals similarities between images in a set. Begin with an image dataset that interests you. Using some sort of image analysis library, such as a convolutional neural network, calculate high-dimensional numeric descriptions for each image. Simplify these description vectors to just two dimensions using a dimensionality reduction algorithm, such as UMAP or t-SNE. Plot your images in the (x,y) locations produced by this algorithm. Discuss the clusters you observe. (Image: a UMAP plot by Christopher Pietsch showing the OpenMoji emoji collection.)



Environmental Sound Clock

Collect some audio recordings of the ambient sound in your room in the morning, at midday, in the evening, and at night. Train a classifier or regressor with these sounds. Use this system to display an approximate estimate of the time.



Sentiment Analysis

Collect the last ten text messages that you sent. Use a sentiment analysis tool to assess the mood of each message.⁶

Additional References

Yining Shi, syllabus for Machine Learning for the Web (NYU ITP, Fall 2018 and Spring 2019), <https://github.com/yining1023/machine-learning-for-the-web>.

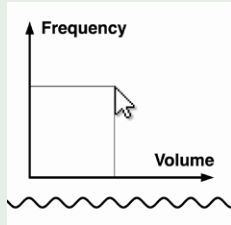
Gene Kogan, "Machine Learning for Artists," GitHub, accessed April 11, 2020, <https://ml4a.github.io/>.

See RunwayML and their learning section that includes libraries for common creative coding environments like Processing. "Learn," RunwayML, accessed April 11, 2020, <https://learn.runwayml.com/#/networking/examples?id=processing>.

Notes

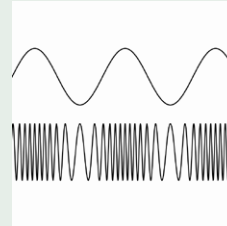
1. See the image classification example in the ml5.js library: "imageClassifier()," accessed April 11, 2020, <https://ml5js.org/reference/api-ImageClassifier/>.
2. See the ml5.js and Teachable Machine projects for in-browser tools to train a model. "Image Classifier," accessed July 11, 2020, <https://learn.ml5js.org/docs/#/reference/image-classifier>; "Teachable Machine," accessed April 11, 2020, <https://teachablemachine.withgoogle.com/>; and Isaac Blankensmith's "ANTI-FACE-TOUCHING MACHINE," <https://twitter.com/Blankensmith/status/1234603129443962880>.
3. Hayk Mikayelyan, "Webcam2Emoji," last modified September 16, 2020, <https://github.com/mikahayk/ml4w-homework2>.
4. Dan Shiffman, "ml5.js Pose Estimation with PoseNet," *The Coding Train*, January 9, 2020, video, 14:24, <https://www.youtube.com/watch?v=Olo-DIOkNVg>.
5. See the featureExtractor regressor in the ml5.js library, and hand trackers such as Ultraleap and Google's MediaPipe HandPose library.
6. See the sentiment model in the ml5.js library: "Sentiment," accessed July 11, 2020, <https://learn.ml5js.org/docs/#/reference/sentiment>.
7. See, for example, RunwayML, or the DCGAN tools in ml5.js.

Sound



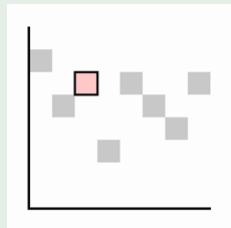
Theremin

A *theremin* is a monophonic instrument whose volume and pitch are controlled by the positions of a player's hands, relative to two antennae. Using the x and y positions of the cursor, create a simple theremin in code. You may not use mouse clicks or the keyboard.



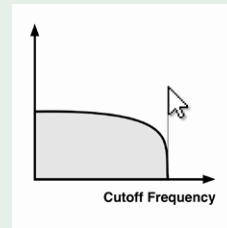
Modulation

Create a project that uses oscillators and envelopes to continuously modify some parameter of a sound. For example, you might implement *tremolo* (periodic amplitude modulation), *vibrato* (periodic frequency modulation), or *wah-wah* (periodic modulation of a resonant filter).



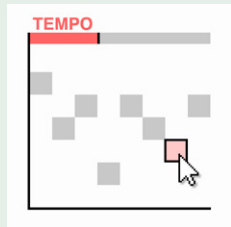
Sequencer I

Create a sequencer that plays a fixed, looping phrase or a collection of sounds. You will need to use arrays to store values that are interpreted as musical notes, chords, rhythms, or other musical parameters. Advanced students: consider how your sequencer's "score" is displayed.



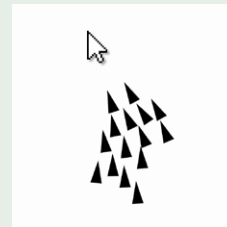
Filter

Create a simple interactive program that permits the user to filter a sound or noise source, using the x and y positions of a cursor to control the filter. Experiment with different types of filters and their parameters.



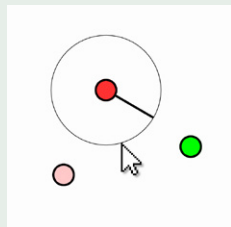
Sequencer II

Create a sequencer that allows a performer to record and play back a looping musical phrase. Devise a visual interface so the performer can alter its playback: for example, changing its speed, transposing it, playing it backwards, reordering its notes, or altering its timbre.



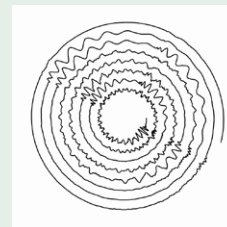
Polyphony

Create a sonic experience that explores polyphony. Scaffold this on a visual system (such as a simulation or game) where the pitch of each voice responds to the speed, position, or orientation of a corresponding particle or mob. For simplicity, restrict your polyphony to multiples of the same instrument or sound.



Sampler

A *sampler* is an electronic instrument that triggers sound recordings. Create a sampler that triggers sounds with key presses. Develop a method for altering the nature of the playback in a way that is coupled to the x and y position of the cursor.



Visualizer

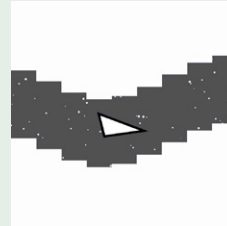
Create a music visualizer for a specific musical recording. Your display should respond to the dynamics (volume changes) and, if possible, the frequency content of your chosen music. For simplicity, be sure to select a recording without speech or lyrics.

Latitude: -51.6221
Longitude: 66.0690



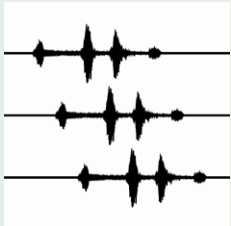
Data Sonification

Locate a real-time Internet data stream, such as one published by a sensor or an API (monitoring weather, stock prices, satellite locations, tweets, seismic events). Create a program that sonifies this data or uses it to control a musical parameter for a generative composition.



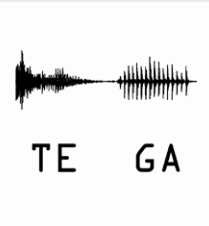
Whistle Cursor

Find a tool to estimate the pitch of a real-time monophonic audio signal, such as the `fzero~` object (Max/MSP/Jitter), `sigmund~` (Pure Data), or `ofxAudio` (openFrameworks). Create an interactive game in which the pitch of the user's whistling is used to steer a spaceship up or down.



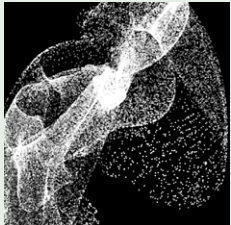
Delay Line Effect

Using a *delay line* (a collection of time-delayed copies of a sound), create a program that transforms a specific sound in a customized way. Popular delay line effects include echo and reverb but also flangers, chorus, and harmonization, when governed by an oscillator.



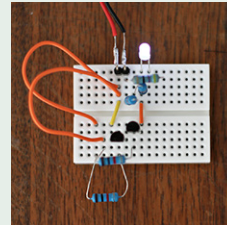
Synthetic Voice

Build a DIY speech synthesizer. Use any audio techniques you wish, except for preexisting speech samples or a text-to-speech system. You could synthesize different vowels with formant synthesis and consonants with gated noise. What word does your machine pronounce best?



Physics-Based Modulation

Create or repurpose some code for a real-time physical simulation, such as a particle system or spring mesh. Use time-varying quantities from the physical simulation to govern continuous parameters of an audio synthesizer. Consider using statistical properties of the simulation as the basis for audio mappings. For example, the average horizontal position of a collection of particles could be mapped to the stereo position of a sound, while the particles' mean velocity could be mapped to pitch. Which mappings produce the most compelling results? Do you note a tight connection between the visualization and sonification of the simulation?



Sound to Physical Action

Use a microphone to control an actuator in a sculpture, environment, or product prototype. Consider how mic placement affects the sounds you'll collect. An example: collect door-knocking sounds with a contact mic; use these to govern a solenoid that makes tapping sounds far away.

Note

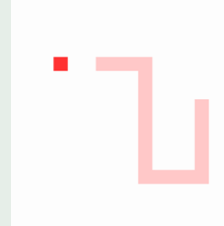
Exercises in this section were contributed by R. Luke DuBois.

Games



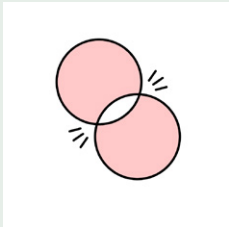
Immaterial Game

Design a game that requires no materials, such as one you might play on a car ride or on a long walk. Your game should require at least two players.¹



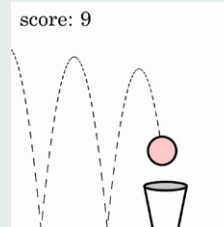
Recoding a Classic

Recreate a classic arcade game such as Pong, Snake, Tetris, or Space Invaders. Alternatively, *modify* a classic arcade game (code for which is readily found); your mod should introduce a twist or subvert the game in some way.³



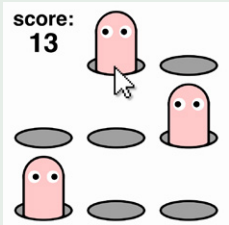
Collision Detection

Write code that moves two circles around the screen in an unpredictable way. (Consider using something like Perlin noise.) When they overlap, have them indicate this by changing color.



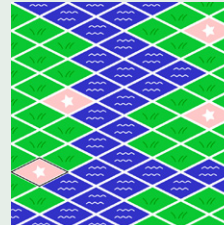
Physics Fun

Create or borrow the code for an organic-looking simulation, such as a bouncing ball or a springy particle system. Now turn it into a game.⁴



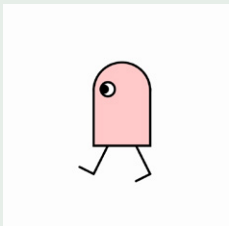
Whack-a-Mole

Create a game in which animals or objects appear and disappear abruptly on the screen, and a player has to click on them to score points.²



Level Designer

Design a data format to represent maps in a dungeon game. Use a system of tiles so that each map combines tiles of different kinds (e.g., walls, paths, water, treasure). Create a "level design tool" that allows a user to choose, place, and save map data.⁵



WASD Navigation

Implement a control system for the movement of an avatar using the WASD keyboard keys.

Notes

1. Dushko Petrovich and Roger White, eds., *Draw It with Your Eyes Closed: The Art of the Art Assignment* (Paper Monument, 2012), 28.

2, 3, 4, 5. Contributed by Paolo Pedercini.

